

Chapter 14

CS2 course package - University of Nebraska-Lincoln (UNL)

Chris Bourke¹

¹University of Nebraska-Lincoln

Abstract

A new lab was created and an old lab was updated in a CS2 course (CSCE 156) at the University of Nebraska–Lincoln in order to introduce PDC concepts. The revised lab introduces a parallel merge sort and has students benchmark it against several sequential sorting algorithms. The new lab introduces connection pooling in a multithreaded application to demonstrate speed up.

Descriptor Elements

Programming Language Employed: The labs utilize Java, JDBC, and SQL.

Relevant core courses: Computer Science 2 (CS2)

Pre-requisites: Students must have completed a CS1 or received credit for CS1 which includes mastery of at least one high-level programming language.

Relevant PDC topics:

Relevant PDC topics and Blooms levels

PDC Concept	Blooms Level
Why and what is parallel or distributed computing?	Remember
Speedup (Parallel and Distributed Models and Complexity	Understand

Learning outcomes:

1. Students should understand the complexities involved in a distributed system and how to handle those complexities.
2. Students should have an appreciation of the speed up that parallel computing can provide.

Context for use: The CS2 course at UNL (CSCE 156) covers Java (as a second programming language), OOP, SQL and databases, list-based data structures, stacks, queues, binary search trees, and heaps. There are several CS1 courses that feed into this course and so students have a variety of programming language background. This is a project-based course; most of the assessment is based on a prescribed database-backed project that is developed iteratively over the course of the semester.

14.1 Introduction

One new lab was created and one current lab was modified to introduce PDC concepts into a CS2 course at UNL. The new lab consisted of a connection pooling project and an old sorting lab was updated to include a parallel mergesort algorithm for benchmarking.

14.2 How CS1 was changed and PDC topics integrated

Minimal modifications were necessary. The sorting lab had already existed and code was provided to students to benchmark. The new lab was introduced in a week that had not had an associated lab for a while (prior to this, there was a lab on recursion that was cut due to duplication from CS1).

14.2.1 Integrated Syllabi (Pre and Post)

No changes were made to the syllabus, PDC related activities were integrated as complementary to existing topics and activities.

14.3 New Unplugged Activities

No unplugged activities were developed or adopted. This course is run in a hybrid/asynchronous manner and has large enrollments, so unplugged activities are not easily integrated.

14.4 New Plugged-in Activities

14.4.1 Connection Pooling

Problem Description, Prerequisites, and Learning Outcomes

One of the primary learning outcomes of CS2 is familiarity with database design, SQL, and database connectivity. Typically students build a substantial database-backed applications, but these are usually single threaded applications.

To expose students to a more realistic distributed database system (as well as load testing and multithreaded applications), we developed a connection pooling lab. The lab uses multiple threads to insert random records into a remote database. To facilitate multiple threads, a connection pool is created using a blocking queue to store and reuse database connections. Students complete the lab by implementing code to retrieve and return database connections using the pool rather than opening/closing a new one every time. Students run several simulations with a configurable number of threads and random records in order to witness the speed up that multiple threads (and using connection pooling) offers.

Algorithms

A consumer-producer pattern is introduced to manage multiple database connections in a thread-safe shared resource pool (Java's `BlockingQueue`). Thread workers are used to simulate a configurable load of inserting multiple records into a database.

14.4.2 Parallel Sort

Problem Description, Prerequisites, and Learning Outcomes

Students are introduced to basic sorting algorithms in CS1, but revisit these topics in CS2 in the context of algorithms and algorithm analysis.

Students complete several implementations of basic sorting algorithms (insertion sort, quick sort) as well as a custom Java `Comparator` to order geographic location objects. They then run several benchmarks and observe how much longer each algorithm takes to sort larger and larger sets of data. This lab was updated to include a parallel merge sort (using Java's built-in `Arrays.parallelSort`).

Algorithms

Java's `Arrays.parallelSort` implementation is a hybrid merge sort algorithm. If the array is less than a certain threshold it defaults to Tim sort, otherwise it uses a `ForkJoin` common pool to break the array into subarrays and sort them in parallel.

Implementation Details

The lab writeups along with all source code is available on github:

- <https://github.com/cbourne/PDCPooling>
- <https://github.com/cbourne/CSCE156-Lab13>

Experience and Teaching Tips

Both labs are heavy on concepts, but light on coding required of students, making these labs easily accessible to most CS2 students. For the connection pooling lab, however, students will need a prior background in SQL and access to an SQL database, ideally remotely in order to provide an observable difference when benchmarking.

Data and Analysis

The common pre/post surveys (See Section 10..7) were administered during the spring 2025 semester. The results are reported in Table 14.1.

14.5 Course-wide Pre and Post Course Surveys

See Section 10..7.

14.6 Other Activities and Ideas

Students could learn the same concepts using any consumer-producer pattern other than using it for database connection pooling. For example, any resource-intensive task (processing whole images, search, etc.) on a sufficiently large set of data lends itself to easy parallelization. Using a queue to store tasks and worker threads conveys the same ideas.

14.7 Conclusion

Though CS2 is less standardized than CS1, it is still possible to introduce PDC related concepts in a way that complements topic coverage. For example, CS2 courses typically cover queue data structures. It is an easy extension to demonstrate a use case for (thread safe) queues as an added PDC topic.

Table 14.1: Survey Results for CS1 - UNL. Sample size was $n = 33$ and included data from spring 2025. The pre- and post means are reported along with the difference (Diff). Statistical significance is measured using a non-parametric Wilcoxon signed-rank (paired) test. The Wilcoxon statistic (W) is reported along with the p -value. Significance is indicated as Yes using $p < 0.05$. Effect size is reported using Cliff's Delta (δ). A negative value indicates an increase in the post-survey data. The interpretation of the effect size is reported (negligible, small, medium, large) as suggested by [1].

Item	Pre	Post	Diff	W	p -value	Sig?	Cliff's δ	Interp.
Computing Topics								
Classes/Objects	3.88	5.42	1.55	5.0	0.000	True	-0.52	large
Lists	3.79	4.91	1.12	21.0	0.000	True	-0.45	medium
Generics	1.76	4.30	2.55	3.5	0.000	True	-0.87	large
Stacks	2.36	4.39	2.03	26.5	0.000	True	-0.70	large
Queues	1.88	4.30	2.42	5.5	0.000	True	-0.80	large
Sorting Algos	3.42	4.67	1.24	3.5	0.000	True	-0.46	medium
Comparators	3.30	4.76	1.45	9.0	0.000	True	-0.45	medium
Recursion	3.70	4.91	1.21	25.0	0.000	True	-0.46	medium
Maps	2.09	4.97	2.88	3.5	0.000	True	-0.82	large
Sets	2.36	4.52	2.15	18.0	0.000	True	-0.69	large
Trees	2.52	4.33	1.82	24.0	0.000	True	-0.64	large
BST	2.39	4.12	1.73	30.5	0.000	True	-0.64	large
Priority Queues	1.64	3.79	2.15	13.5	0.000	True	-0.78	large
Hashing	1.61	3.52	1.91	25.0	0.000	True	-0.77	large
Graphs	2.76	4.00	1.24	105.5	0.003	True	-0.44	medium
Unit Tests	3.00	5.03	2.03	6.0	0.000	True	-0.62	large
Big-O	1.52	4.39	2.88	6.5	0.000	True	-0.89	large
DS Choices	3.03	4.55	1.52	18.0	0.000	True	-0.53	large
Designing/Writing Classes	3.27	5.52	2.24	5.5	0.000	True	-0.67	large
Decomposition	4.33	5.24	0.91	45.5	0.002	True	-0.33	medium
PDC Topics								
Parallel	2.45	3.39	0.94	75.5	0.005	True	-0.36	medium
Distributed	2.55	3.88	1.33	65.0	0.003	True	-0.45	medium
Event	3.24	3.48	0.24	163.5	0.754	False	-0.11	negligible
Attitudes								
Interest in Com- puting	6.27	5.79	-0.48	0.0	0.007	True	0.23	small
Interest in PDC	5.58	5.12	-0.45	30.0	0.041	True	0.16	small
Learning PDC is Important	6.03	5.76	-0.27	28.5	0.116	False	0.19	small
Understanding of Computing	4.88	5.21	0.33	81.5	0.065	False	-0.16	small
Understanding of PDC	3.42	4.55	1.12	56.5	0.002	True	-0.35	medium

Bibliography

- [1] Jeanine Romano, Jeffrey D Kromrey, Jesse Coraggio, and Jeff Skowronek. Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the NSSE and other surveys. Presented at the annual meeting of the Florida Association of Institutional Research, 177, 2006.

Appendix

|You must include these four specific subsections (in addition to anything else):|

Github Link to Instructional Material

- Connection Pooling Lab: <https://github.com/cbourne/PDCPooling>
- Sorting Lab: <https://github.com/cbourne/CSCE156-Lab13>

<https://github.com/...>

Detailed Pre and Post Syllabus

No changes were made to the syllabus.